

ROOT講習会第5回 TTree / データ構造

東大宇宙線研 M2 杉本布達

2025/07/02

自己紹介

名前：杉本 布達（Discord上では fsugim です）

所属：東大宇宙線研 Tibet AS γ / ALPACA グループ

研究内容：太陽活動と宇宙線量の関連など

先週は体調を崩してしまいすみません、、、

ボリビアの実験サイトにて

検出器



おさらい

これまで、以下のようなことを学んだと思います

- マクロファイルの実行
 - プロット (TGraph) / ヒストグラム (TH1D etc.) を作る
 - 関数 (TF1)でのフィッティング
- … なにかがたりない？ **データを保存したい！**

データ保存について

例えばこんなときに処理後のデータを保存したい：

- ・元データが動画などで容量を食いすぎる（データ圧縮）
- ・特定の処理をしたあと（フィット後の結果など）

というモチベーションがあります。

テキストでもいいですが、**テキストよりも早い方法**があります。

おまけ：なんでテキストより早いようなファイルがあるか考えてみましょう。

TTree

テキスト読み込みより早いのが、ROOTのTTreeです。
ざっくり使用方法を見てみましょう。

- TTreeでは、データは表のように格納されています。
 - それぞれの列では、データの型が決まっています。
 - 実際のプログラム中では、intならI, doubleならDなどと表記します。

Event number (int)	天頂角 (double)	方位角 (double)	粒子数 (int)	ミューオン数 (int)
0	1.025...	1.786...	13	0
1	0.034...	0.872...	45	6
2	0.591...	5.839...	95	15
3	0.130...	0.435...	27	2
...	...	...	...	...

TTree

ということで、手を動かしていきましょう！
まず、rootファイルをダウンロードします。

```
[fsugim@icrhome04 root_lecture]$ root -l sim_shower_events.root  
root [0]  
Attaching file sim_shower_events.root as _file0...  
(TFile *) 0x2794100  
root [1] █
```

上のようなコマンドでデータを読んでみます。

表示される (TFile *) ... が 0x0 でなければOKです。

補足：0x は16進数を表します。0x0 はヌルポインタ(= NULL)と呼ばれ、実体がないことを指します。

TTree

ということで、手を動かしていきましょう！
まず、rootファイルをダウンロードします。

- `sim_shower_events.root`
- `sim_shower_events_gamma.root` の2種類あります。

レクチャー部分では `sim_shower_events.root` のみ使います

補足：0x は16進数を表します。0x0 はヌルポインタ(= NULL)と呼ばれ、実体がないことを指します。

TTree 次に、データ一覧を確認してみましょう。

.ls で表示される”shower”
が今回のTTreeの名前です

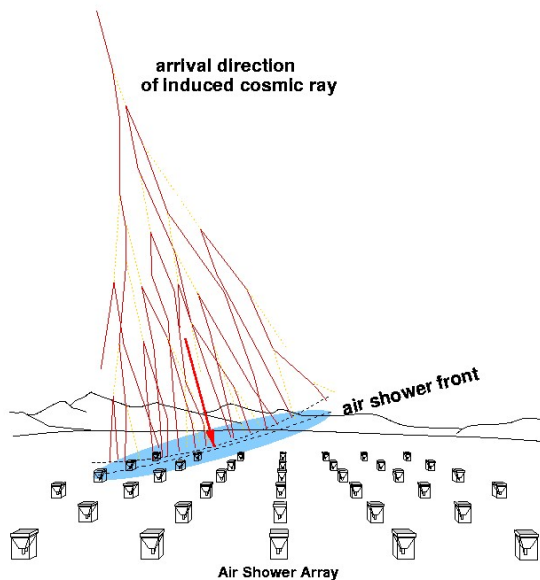
shower->Print()で、中に入っているデータを確認
することができます。

```
[fsugim@icrhome04 root_lecture]$ root -l sim_shower_events.root
root [0]
Attaching file sim_shower_events.root as _file0...
(TFile *) 0x304e4a0
root [1] .ls
TFile**          sim_shower_events.root
TFile*           sim_shower_events.root
KEY: TTree      shower;1          Simulated Shower Events
root [2] shower->Print();
*****
*Tree   :shower   : Simulated Shower Events *
*Entries : 1000000 : Total =      16050632 bytes File Size =    9423852 *
*       :         : Tree compression factor =    1.70 *
*****
*Br    0 :zenith   : zenith/F *
*Entries : 1000000 : Total Size=    4012268 bytes File Size =    3584961 *
*Baskets :    126 : Basket Size=    32000 bytes Compression=    1.12 *
*.....*
*Br    1 :azimuth  : azimuth/F *
*Entries : 1000000 : Total Size=    4012398 bytes File Size =    3584852 *
*Baskets :    126 : Basket Size=    32000 bytes Compression=    1.12 *
*.....*
*Br    2 :num_particle : num_particle/I *
*Entries : 1000000 : Total Size=    4013048 bytes File Size =    1324527 *
*Baskets :    126 : Basket Size=    32000 bytes Compression=    3.03 *
*.....*
*Br    3 :num_muon  : num_muon/I *
*Entries : 1000000 : Total Size=    4012528 bytes File Size =    924345 *
*Baskets :    126 : Basket Size=    32000 bytes Compression=    4.34 *
*.....*
root [3]
```


ちょっとだけ研究グループの紹介

データの内容について説明するために、ちょっと脇道へそれます

Tibet AS γ / ALPACA グループは空気シャワーを観測しています



空気シャワーは、大気中に入射した粒子が空気中の粒子と反応することで、二次粒子のカスケードを作る現象です。

二次粒子を地上で観測することで、元の粒子の到来方向がわかる、という仕組みです。

演習 - データの確認

ということで、rootファイル中のデータの列はそれぞれこんな感じになっているはずです

列名	データ型	意味
Zenith	Double	空気シャワーの天頂角
Azimuth	Double	空気シャワーの方位角
Num_particle	Int	観測された粒子数
Num_muon	Int	観測されたミューオン数

今回は (自分が書いた)シミュレーションなので、天頂角・方位角を一様確率で振っています。まずその確認をしましょう。

演習 – TTreeのデータの読み方

TTreeの各列のことを**Branch**と呼びます。枝だからですね。

TTree名->Draw(“Branch名”)で、その列のデータのヒストグラムを描くことができます。

例えば、方位角 (“zenith”)のヒストグラムを描いてみます。

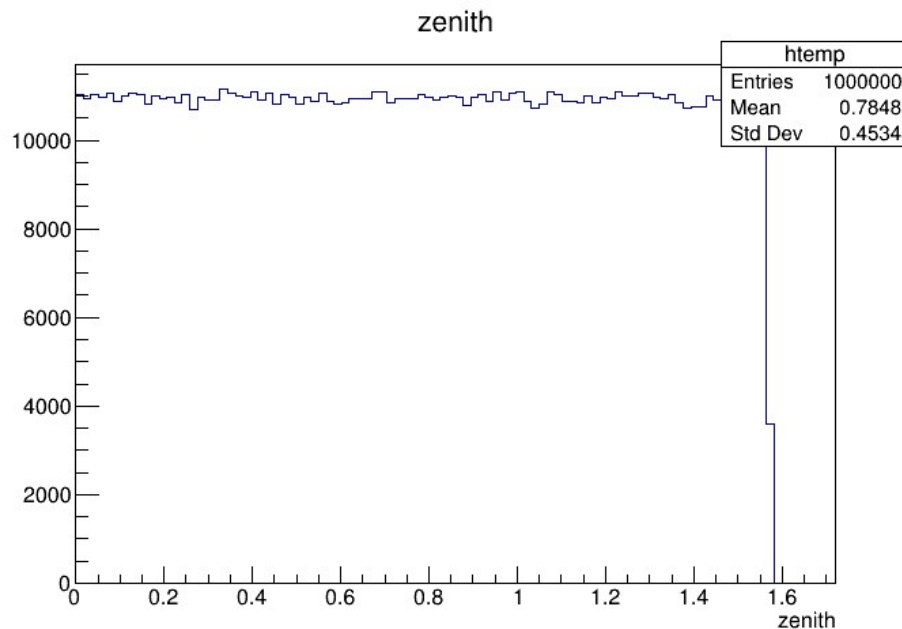
```
root [1] shower->Draw("zenith");  
Info in <TCanvas::MakeDefCanvas>:  created default TCanvas with name c1
```

このようなコマンドを打つと、画面が現れると思います。

演習 – TTreeのデータの読み方

例えば、天頂角 (“zenith”) のヒストグラムを描いてみます。

```
root [1] shower->Draw("zenith");  
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```



このような形になります。

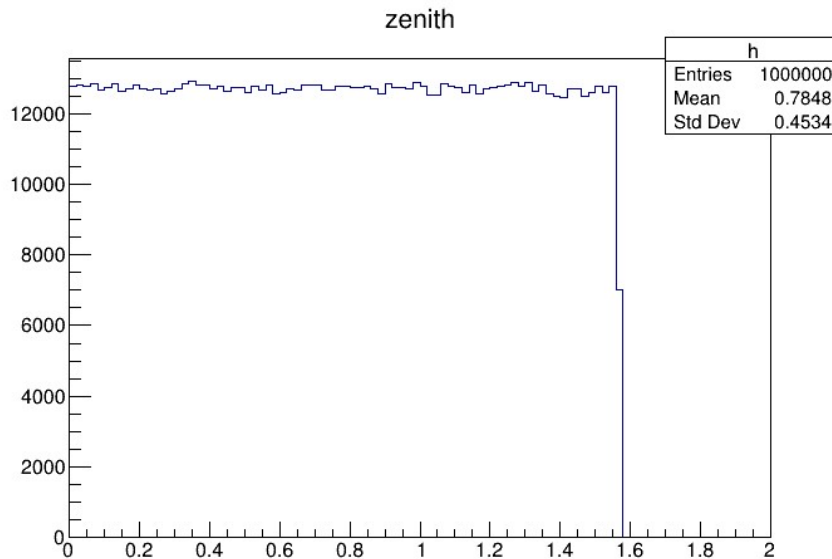
横軸はradなので、 $0 - \pi/2$ まで等方に振れていそうです。

ただ、Legendにデータが隠れてしまっています。

演習 – TTreeのデータの読み方

Branch名の後に>>をつけて、ビン数、最小値、最大値を指定できます。ビン数を100, 最小値を0, 最大値を2にしてみましょう。

```
root [9] shower->Draw("zenith>>h(100,0,2)");  
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```



このような形になります。

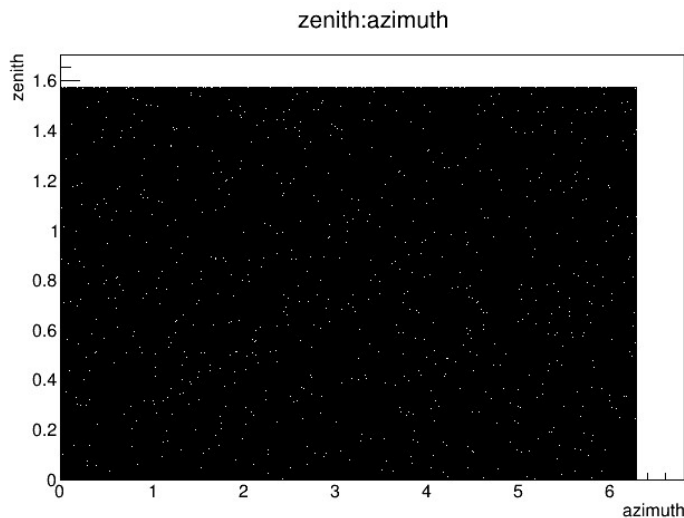
今度は、ほぼ同じくらいデータが入っているのが見えました。

演習 – TTreeのデータの読み方

同様に、2次元のヒストグラムも描いてみましょう。

今度は、`shower->Draw("zenith:azimuth")`のようにbranchをコロンを挟んで二つ指定します。

```
root [5] shower->Draw("zenith:azimuth");  
Info in <TCanvas::MakeDefCanvas>:  created default TCanvas with name c1
```



このような形になります。

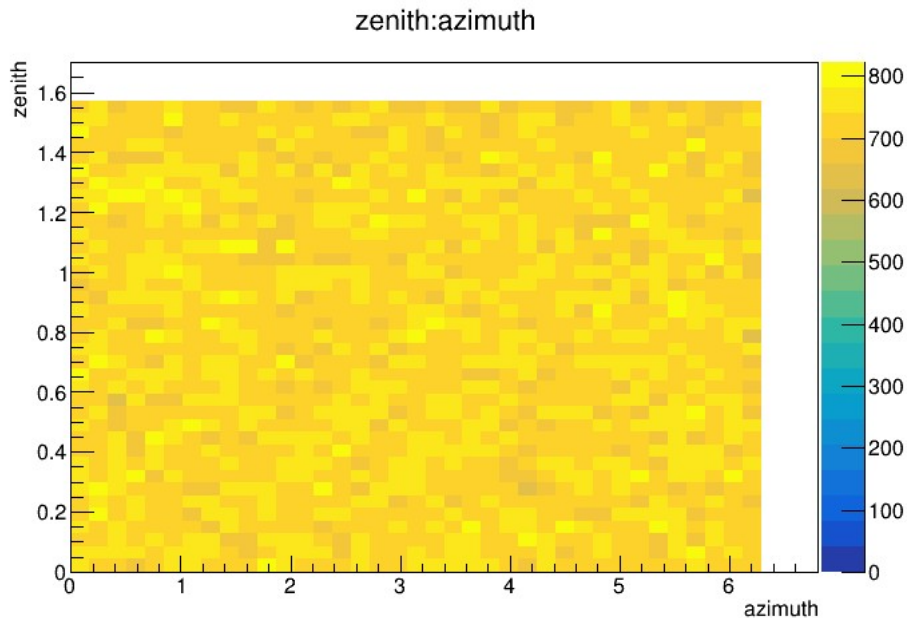
真っ黒ですね…！
もう少し見やすくしてみます。

演習 – TTreeのデータの読み方

Drawという関数の第三引数で、描画形式を指定しましょう。

shower->Draw("zenith:azimuth", "", "**COLZ**") と書き換えます。

```
root [6] shower->Draw("zenith:azimuth", "", "COLZ");  
Info in <TCanvas::MakeDefCanvas>:  created default TCanvas with name c1
```



すると、今度はビンで区切られたデータが出力されました。

天頂角、方位角ともにほぼ一様にデータが振られていそうです。

演習 – TTreeのデータの読み方

さて、天頂角と方位角はほぼ一様に振れていることが分かったと思います。

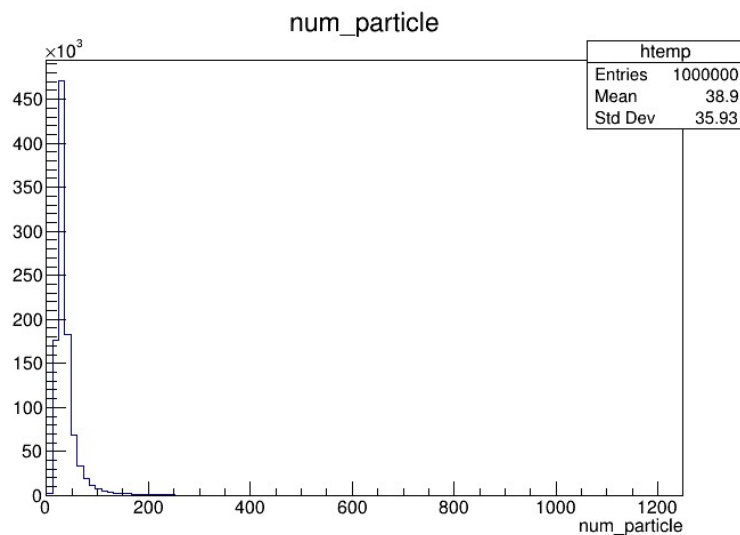
そこで、粒子数・ミューオン数の一次元分布を見てみましょう。

粒子数のBranch名は”num_particle”、ミューオン数のBranch名は”num_muon”でした。先ほどの例を参考にしつつ、実際に手を動かしてみてください。

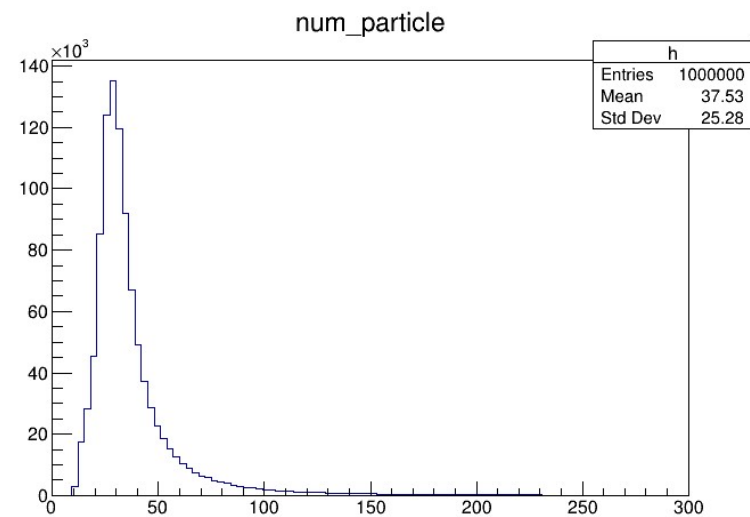
演習 – TTreeのデータの読み方

おさらいですが、TTree名 -> Draw(“Branch名”) でヒストグラムを描画できるのでした。

```
root [15] shower->Draw("num_particle");  
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```



単純に描いた

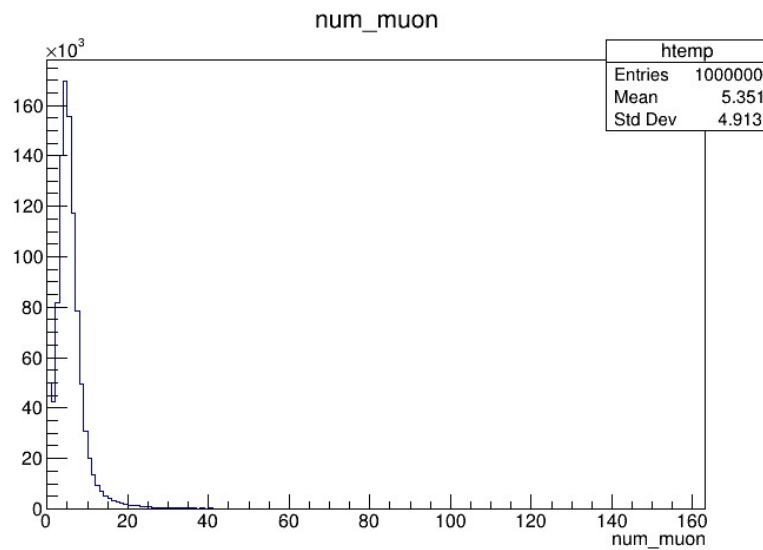


描画範囲を狭めた

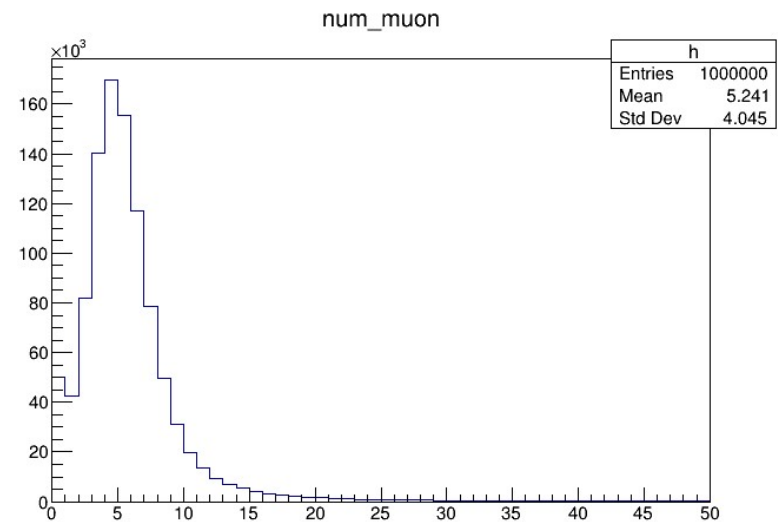
演習 – TTreeのデータの読み方

おさらいですが、TTree名 -> Draw(“Branch名”) でヒストグラムを描画できるのでした。

```
root [21] shower->Draw("num_muon");  
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```



単純に描いた



描画範囲を狭めた

演習 – TTreeのデータの読み方

粒子数・ミューオン数の一次元分布を見てみましたが、何かの特徴に気づきましたか？

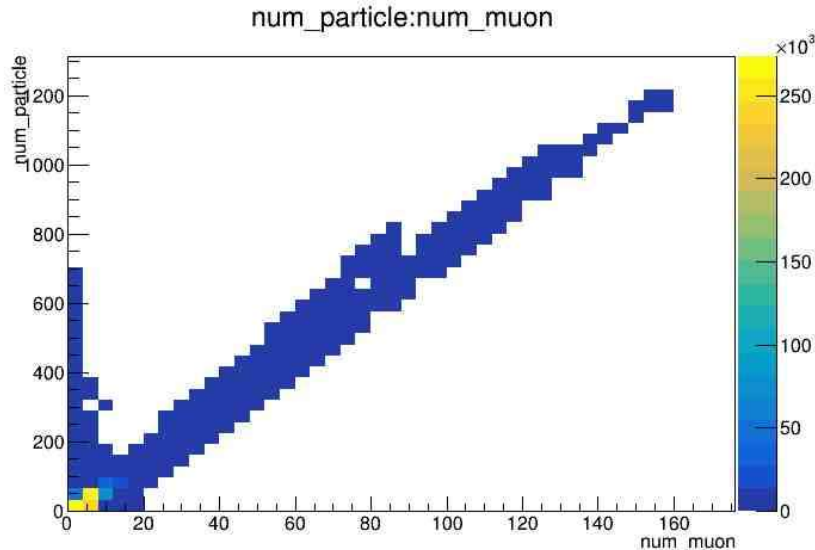
なかなか難しいそうですね。。。二次元分布にして特徴を見てみましょう。

二次元分布を描くコマンドは、`Draw(“branch1:branch2”, “”, “COLZ”)`という形式なのでした。

演習 – TTreeのデータの読み方

二次元分布を描くコマンドは、`Draw("branch1:branch2", "", "COLZ")`という形式なのでした。

```
root [1] shower->Draw("num_particle:num_muon", "", "COLZ");  
Info in <TCanvas::MakeDefCanvas>: created default TCanvas with name c1
```



なんと、データに明確な傾向が見えました！

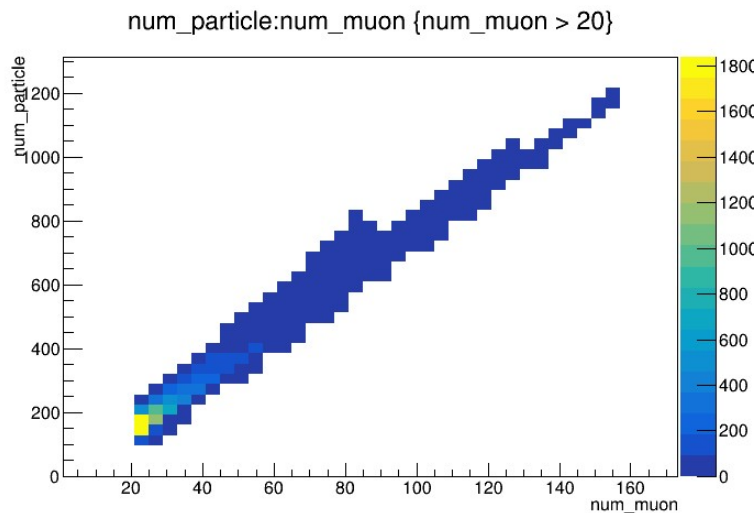
粒子数とミューオン数が比例して見えるデータと、そうでないものが混ざっているように見えます。

演習 – TTreeのデータの読み方

とりあえずカットをかけてみましょう。

Drawの第二引数にカット条件を書くことができます。今回は、`num_muon > 20`をかけてみます。

```
root [3] shower->Draw("num_particle:num_muon", "num_muon > 20", "COLZ");  
Info in <TCanvas::MakeDefCanvas>:  created default TCanvas with name c1
```



ちゃんとミュオン数が20以下のデータはカットされているように見えます。

また、この範囲ではミュオン数と粒子数が比例しているように見えます。

演習 – TTreeのデータの読み方

ということで、4つのBranchのデータを取り扱ってきました。
さらに、同じこと+ α をマクロでもやってみましょう。

手元に

- read_root.c
- read_root_mult.c
- data_to_root.c

の3ファイルを用意してください。

演習 – TTreeのデータの読み方

read_root.c

ということで、4つのBranchのデータを取り扱ってきました。
さらに、同じことをマクロでもやってみましょう。

デフォルトで使うファイル名を入れてます。呼び出しで他のファイル名を入れるとそれが参照されます。

```
1 void read_root(TString filename="data/sim_shower_events.root") {  
2  
3     auto file = TFile::Open(filename);  
4     auto tree = dynamic_cast<TTree*>(file->Get("shower"));  
5  
6     double zenith, azimuth;  
7     int num_particle, num_muon;  
8  
9     tree->SetBranchAddress("zenith", &zenith);  
10    tree->SetBranchAddress("azimuth", &azimuth);  
11    tree->SetBranchAddress("num_particle", &num_particle);  
12    tree->SetBranchAddress("num_muon", &num_muon);
```

rootファイルを開きます。::はクラスのメソッド呼び出しです（あとで少し触れます）。

ポインターをTTreeのポインターに変換します。dynamic_castは変換の安全な方法です。

それぞれのBranchに対応する変数を紐づけます。&(variable) はその変数のアドレスを与える演算子なのです。

演習 – TTreeのデータの読み方

read_root.c

```
14 long long entries = tree->GetEntries();
15
16 const int nbin = 100;
17 const int val_max = 200;
18 const int val_min = 200;
19
20 auto hist = new TH1D("hist", "hist", nbin, val_min, val_max);
21 hist->SetTitle(";");
22
23 for (int i = 0; i < entries; i++) {
24     tree->GetEntry(i);
25     hist->Fill(num_particle);
26 }
27 hist->Draw();
28
29 }
```

tree-> GetEntry(i)をすると、i番目のデータが取得されます。データは紐づけた変数に書き込まれます。
Num_particle, zenith... の値が更新されるということですね。
cout << num_particle << endl; を追記すると、カウンター変数 i が進むごとに値が変わるはずですよ。

演習 – TTreeのデータの読み方

`read_root_mult.c`

では、もう一つのrootファイルと合わせて解析を試みましょう。

rootファイルのTTreeからTH1Dにデータを入力する部分を関数として切り出します。

```
1 TH1D * read_data(TString filename="data/sim_shower_events.root") {
2
3     auto file = TFile::Open(filename);
4     auto tree = dynamic_cast<TTree*>(file->Get("shower"));
5
6     float zenith, azimuth;
7     int num_particle, num_muon;
8
9     tree->SetBranchAddress("zenith", &zenith);
10    tree->SetBranchAddress("azimuth", &azimuth);
11    tree->SetBranchAddress("num_particle", &num_particle);
12    tree->SetBranchAddress("num_muon", &num_muon);
13
14    long long entries = tree->GetEntries();
15
16    const int nbin = 100;
17    const int val_min = 0;
18    const int val_max = 200;
19
20    auto hist = new TH1D("hist", "hist", nbin, val_min, val_max);
21    hist->SetTitle("");
22
23    for (int i = 0; i < entries; i++) {
24        tree->GetEntry(i);
25        hist->Fill(num_particle, num_muon);
26    }
27
28    return hist;
29
30 }
```

演習 – TTreeのデータの読み方

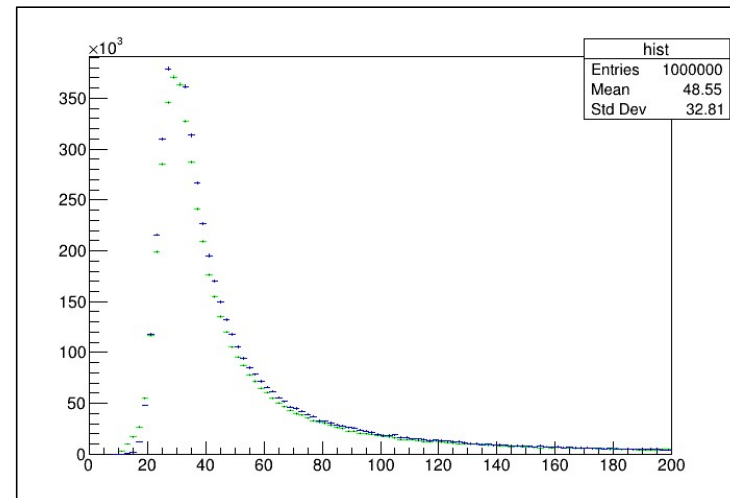
read_root_mult.c

この関数を使って、複数のrootファイルからデータを取得します。

その後、TH1DをそれぞれDrawします。

右下の図のようなヒストグラムが描かれるはずです。

```
32 void read_root_mult() {  
33  
34     auto hist1 = read_data("data/sim_shower_events.root");  
35     auto hist2 = read_data("data/sim_shower_events_gamma.root");  
36  
37     hist1->SetLineColor(kGreen);  
38     hist1->Draw();  
39  
40     hist2->SetLineColor(kBlue);  
41     hist2->Draw("SAME");  
42 }
```



演習 – TTreeの作り方

(このスライドは座学として聞いてください)

rootファイルにデータを保存するには、新しいTTreeを作成し、tree->Fill()でデータを埋め込んでいきます。

右の例は、csvファイルからデータを読み込むサンプルです。

データが膨大な場合、TChainでツリーを複数ファイルに分けて扱うこともできます。

```
1 void csv_to_root()
2 {
3     // 入力ファイル名
4     const char* infile = "sim_shower_events_gamma.csv";
5     const char* outfile = "sim_shower_events_gamma.root";
6
7     // 出力ROOTファイル
8     TFile* f = new TFile(outfile, "RECREATE");
9     TTree* tree = new TTree("shower", "Simulated Shower Events");
10
11     // 変数定義
12     float energy, zenith, azimuth;
13     int num_particle, num_muon, label, composition;
14
15     tree->Branch("zenith", &zenith, "zenith/F");
16     tree->Branch("azimuth", &azimuth, "azimuth/F");
17     tree->Branch("num_particle", &num_particle, "num_particle/I");
18     tree->Branch("num_muon", &num_muon, "num_muon/I");
19
20     // ファイルを開く
21     std::ifstream fin(infile);
22     // 4 lines: if (!fin.is_open()) { .....
23
24     std::string line;
25     // 1行目はヘッダーなので読み飛ばし
26     std::getline(fin, line);
27
28     while (std::getline(fin, line)) {
29         std::istringstream ss(line);
30         std::string val;
31         // カンマ区切りで各カラムをパース
32         std::getline(ss, val, ','); energy = std::stof(val);
33         std::getline(ss, val, ','); zenith = std::stof(val);
34         std::getline(ss, val, ','); azimuth = std::stof(val);
35         std::getline(ss, val, ','); num_particle = std::stoi(val);
36         std::getline(ss, val, ','); num_muon = std::stoi(val);
37         std::getline(ss, val, ','); label = std::stoi(val);
38         std::getline(ss, val, ','); composition = std::stoi(val);
39
40         tree->Fill();
41     }
42
43     tree->Write();
44     f->Close();
45     fin.close();
46 }
```


演習 – TTreeの作り方

rootファイルにデータを保存するには、新しいTTreeを作成し、tree->Fill()でデータを埋め込んでいきます。

rootファイルから読み込んだデータを選別し、新しいrootファイルを作ることもできます。

たとえば、ここではnum_muonの制限をしています。

data_to_root.c

28

```
1 void data_to_root()
2 {
3     // 入力ファイル名
4     const char* infile = "data/sim_shower_events.root";
5     const char* outfile = "sim_shower_events_gamma.root";
6
7     // 入力ROOTファイル
8     TFile* file_in = TFile::Open(infile);
9     auto in_tree = dynamic_cast<TTree*>(file_in->Get("shower"));
10
11     // 出力ROOTファイル
12     TFile* file_out = new TFile(outfile, "RECREATE");
13     TTree* tree = new TTree("shower", "Simulated Shower Events");
14
15     // 変数定義
16     float zenith, azimuth;
17     int num_particle, num_muon;
18
19     in_tree->SetBranchAddress("zenith", &zenith);
20     in_tree->SetBranchAddress("azimuth", &azimuth);
21     in_tree->SetBranchAddress("num_particle", &num_particle);
22     in_tree->SetBranchAddress("num_muon", &num_muon);
23
24     tree->Branch("zenith", &zenith, "zenith/F");
25     tree->Branch("azimuth", &azimuth, "azimuth/F");
26     tree->Branch("num_particle", &num_particle, "num_particle/I");
27     tree->Branch("num_muon", &num_muon, "num_muon/I");
28
29     // ファイルを開く
30     long long entries = in_tree->GetEntries();
31
32     for (int i = 0; i < entries; i++) {
33         in_tree->GetEntry(i);
34         if (num_muon < 20) {
35             tree->Fill();
36         }
37     }
38
39     tree->Write();
40     file_out->Close();
41 }
42 }
```

レクチャー部分終わり

- ・お疲れさまでした！

この後は各自演習です。

先ほどダウンロードしていなかったかもしれない

- ・ `sim_shower_events_gamma.root`

をダウンロードしてください。

演習問題

- TH1Dでのデータ読み込みをTH2Dに拡張してみましょう。
変数の定義が変わるので、binの最小値・最大値をx, y軸分用意する必要があります。

TH2Dの定義はTH2D(“name”, “title”, nbinsx, xbin_min, xbin_max, nbins_y, ybin_min, ybin_max)です！

- 2つのファイルのデータを1つのTH2Dに読み込みましょう。
関数の引数を一つ増やしてみましょう。
- TH2Dのx軸をnum_muon, y軸をnum_particleに設定してデータを確認してみましょう。

演習問題

- 粒子数とミュオン数が比例するデータと、しないデータをカット条件を設けて区別してみましょう。

例えばnum_muonsのような変数の値によってカットをかけてみましょう。

- それぞれ、天頂角分布と方位角分布をプロットしてみましょう。

1Dプロットでもいいですが、できれば2Dプロットにしてみましょう。

プログラムの答えはanswer.cにあります。

演習問題

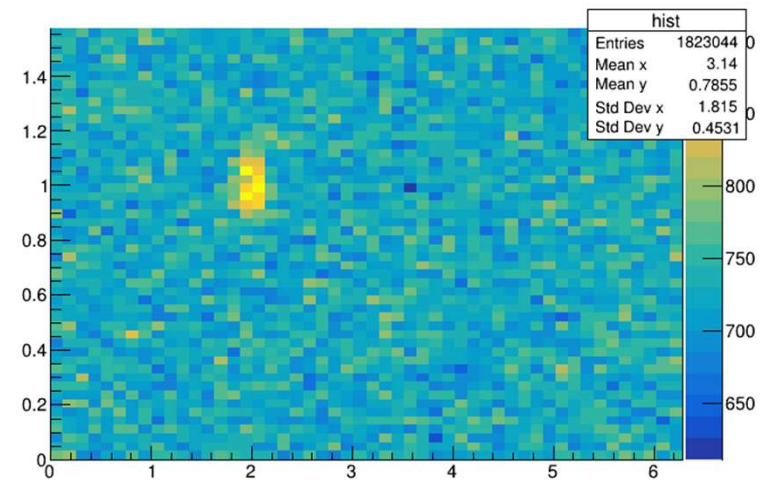
・ $xbin = ybin = 50$ とします。ビンごとの平均データ数をバックグラウンドとして、一番データが多いビンの有意度を

$$S/\sqrt{S+B}$$

で計算してみてください。

演習問題（答え）

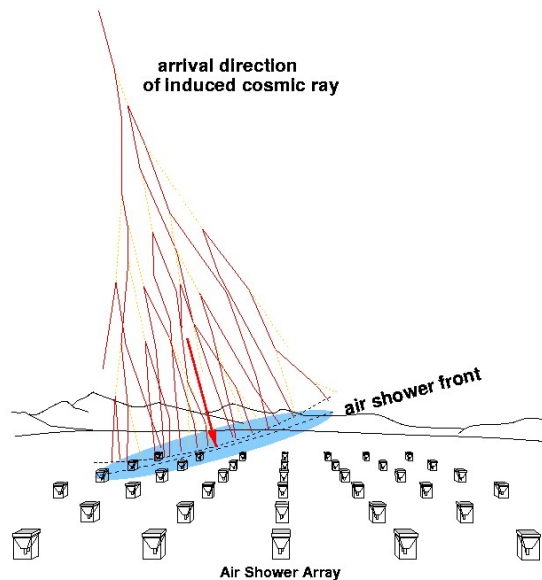
・うまくカットができてると、おおまかにこういう2次元分布になるはず。。。横軸が方位角、縦軸が天頂角です。



- ・ちなみに有意度は $> 5\sigma$ になります。
- ・カットをしない場合も実は見えます。。。 （シミュレーションのせいですが）

おまけ

Tibet AS γ / ALPACA グループは空気シャワーを観測しています
空気シャワーには、主に電磁成分が入射するものと、ハドロン成分が入射するものがあります。



ハドロン（陽子）などが入射するとミュオンを多く作り、電磁成分だとミュオンはほぼ作られないという性質があります。

今回のシミュレーションはこの違いを念頭において作っていたので、ミュオン数でこれらを区別することができた、ということでした。

ミュオン数が少ないのはガンマ線が作るシャワーだと思っていた、ということですね。つまり、ガンマ線を見分けてある方向から来ていることを確認するという行為を演習として用意しました。

C++のクラス関数、型推論 (発展)

TFile::Open はTFileというクラスの関数です。

クラスそのものに紐づいているため、こういう書き方をしています。

`auto f = new TFile(); f->Open("rootfile");`でも別に動くのでok。ただ冗長にはなります。

クラスの関数などはプログラムとかデータの設計をするようになってからの勉強でいいはずです。

dynamic_cast<TTree*>はもとのポインタをTTree*型として解釈するように指示する演算子です。

もともと (double) などとかなどと書くと勝手にdouble型として処理してくれますが、dynamic_castでは実行時に変な値ではないかチェックしてくれます！！

チェックがいない場合はstatic_cast<type> という書き方をすればいいです。これはコンパイル時にチェックしてくれます。

auto variable = new (….) は型を推論して、その型のオブジェクトとして取り扱ってくれるキーワードです。ありがたいのですが、関数定義には使えません。また、変な型になる場合もあります。

C++のクラス関数、型推論 (発展)

ここらへんは完全におまけです。時間があれば。

いま `auto variable = new Ttree...` みたいな書き方をしていますが、これはオブジェクト分メモリを確保しており、明示的にはプログラム終了時にメモリを開放していません。小規模なプログラムではこれでもいいのですが、使い方によってポインタの管理は分けられるべきです。

例えば、`auto variable = unique_ptr<Ttree *>{new Ttree...}` のような書き方をすると、プログラムの終了時に自動でこのポインタによって確保されてるメモリーが解放されます。`shared_ptr`も同様です。違いは検索してみて・LLMに聞いてください。

仕組み上ROOTを使っている範疇では問題ないですが、メモリリークを起こすとOS全体の動きが著しく遅延して問題になります。できるだけお行儀のよい書き方をしましょう。

これらに関しても、詳しくはcppreference.jpなどを参照してください。